FINAL EXAMINATION

Smart Campus Energy and EV Fleet Management

ITS72604 | September 2026 Semester

Candidate detail	Entry
Student Name	LIYUZHOU
Student ID	0398565
Score	For examiner use



Academic answer report and implementation summary

Section A Question 1

Distinct roles of NumPy and Pandas in high-frequency telemetry

NumPy is suited to holding regular, numerical telemetry arrays and performing slicing, masking, and element-wise operations; Pandas provides DataFrames with column names, indexes, and grouping semantics, making it suitable for cleaning, joining, and aggregating by device category. By first using Pandas to preserve business fields and grouping relationships, then applying NumPy's vectorized operations to continuous numerical columns, one can retain both readability and numerical computation efficiency (NumPy Developers, n.d.; pandas Development Team, n.d.).

The example below extracts afternoon peak columns from a two-dimensional 'charging hub x hour' grid-load matrix and uses a boolean mask to locate records where battery temperature exceeds 45C. The Pandas example fills missing Energy_kWh with group medians by Charger_Type, then computes revenue for each charger type.

Code Listing 1. Vectorized NumPy and grouped Pandas operations

```
import numpy as np
import pandas as pd

# Rows represent charging hubs; the four columns represent hours 8, 14, 15, and 16.
grid_load_kw = np.array([
    [18.2, 20.1, 25.4, 22.8],   # Four example hourly load observations
    [14.5, 19.8, 29.1, 24.0],
])
peak_window = grid_load_kw[:, 1:4]  # Slice the afternoon peak hours 14–16.

battery_temp_c = np.array([39.5, 46.2, 44.8, 51.0])
hot_temperature_mask = battery_temp_c > 45.0
high_temperature_values = battery_temp_c[hot_temperature_mask]

# Fill missing energy separately for AC and DC because their distributions differ.
df["Energy_kWh"] = df.groupby("Charger_Type")["Energy_kWh"].transform(
    lambda values: values.fillna(values.median())
)
# Compute a revenue total for each charger category in one grouped operation.
revenue_by_charger = df.groupby("Charger_Type")["Revenue_MYR"].sum()
```

Vectorization delegates array-level operations to an optimized low-level implementation, avoiding the interpreter overhead of row-by-row Python loops; broadcasting can also perform operations on compatible shapes without copying scalar inputs. However, excessively large intermediate arrays still consume memory, so data should be processed by column or batch and only necessary variables retained (NumPy Developers, n.d.). Pandas' groupby adopts a split-apply-combine idiom, keeping grouped statistics close to business dimensions and easy to audit (pandas Development Team, n.d.).

Section A Question 2

Functional processing of high-risk charging sessions

The program first filters alert sessions by two conditions: overstay time strictly greater than 20 minutes AND battery temperature strictly greater than 40C; it then applies 15% of the base energy fee as a surcharge and finally accumulates the surcharge revenue. filter and map return lazy iterators, and reduce accumulates in order, avoiding the creation of entire intermediate lists at each step; for very large production datasets, chunked processing can be adopted instead. The accumulation semantics of reduce are consistent with the Python standard library definition (Python Software Foundation, n.d.).

Code Listing 2. Lazy filtering, surcharge mapping, and reduction

```
from functools import reduce
from typing import Iterable, Mapping, Any

Session = Mapping[str, Any]

def calculate_alert_surcharge(sessions: Iterable[Session]) -> float:
    """Return surcharge revenue for sessions matching both risk rules."""
    high_risk = filter(
        lambda row: row["overstay_minutes"] > 20
        and row["battery_temperature_c"] > 40,
        sessions,
    )
    surcharge_values = map(
        lambda row: row["base_energy_fee_myr"] * 0.15,
        high_risk,
    )
    return reduce(lambda running_total, amount: running_total + amount,
                  surcharge_values, 0.0)
```

A lambda describes only a single, side-effect-free condition or transformation; the filter predicate explicitly connects the two rules with `and`, avoiding the mistake of treating 'either condition' as 'both conditions satisfied'. reduce provides a 0.0 initial value, so it returns a stable 0 even when there are no alert sessions.

Section A Question 3

Executive visualization design and library comparison

Executive reports should prioritize answering 'when, which device type, and how large the difference is', rather than piling on decoration. The hourly revenue bar chart should use consistent, distinguishable, and colorblind-friendly colors for AC and DC, sorted by hour; the correlation heatmap should use a diverging colormap centered at 0 to express positive and negative directions, annotate cells with correlation coefficients, and retain a legend. Titles, axis units, and data-source notes enable non-technical managers to quickly grasp chart meaning.

Matplotlib provides fine-grained control over Figures, Axes, ticks, annotations, and layout, making it suitable for final typesetting and composite charts; Seaborn is built on top of Matplotlib and provides statistical plotting interfaces such as boxplot, heatmap, and regplot, expressing grouping and distribution with less code. In practice, Seaborn can be used to describe data structure, and Matplotlib can then be used to adjust units, annotations, and layout (Matplotlib Development Team, n.d.; seaborn Development Team, n.d.).

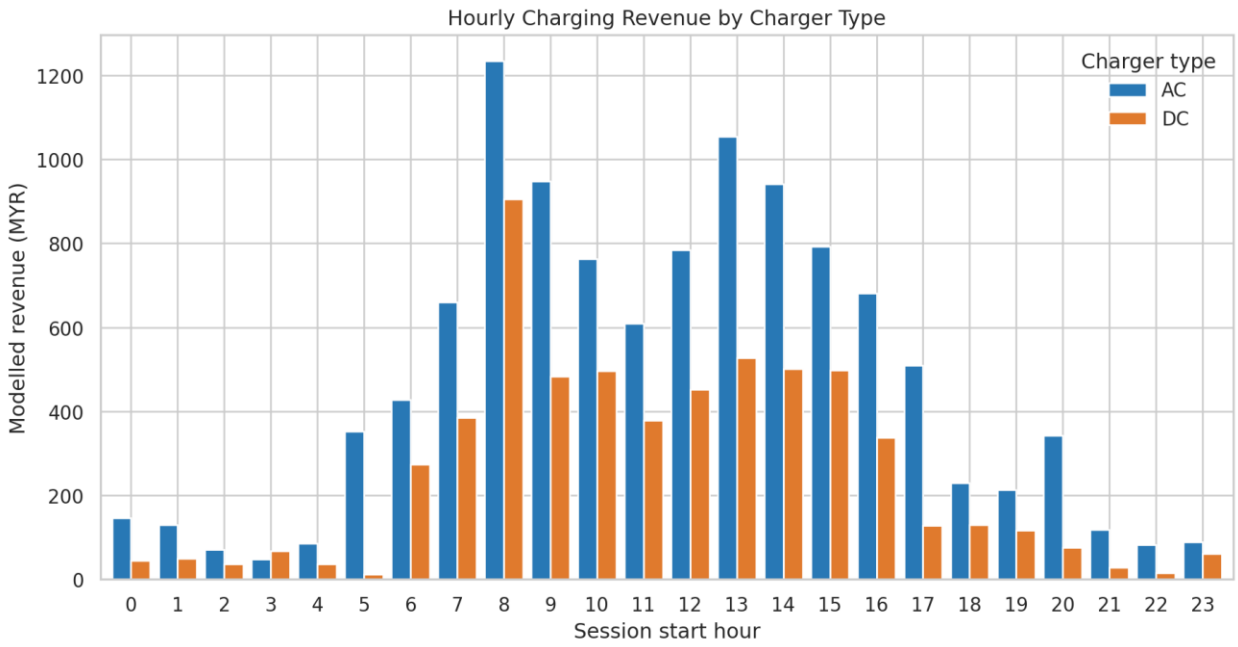


Figure 1. Modelled hourly revenue split by AC and DC charger type.

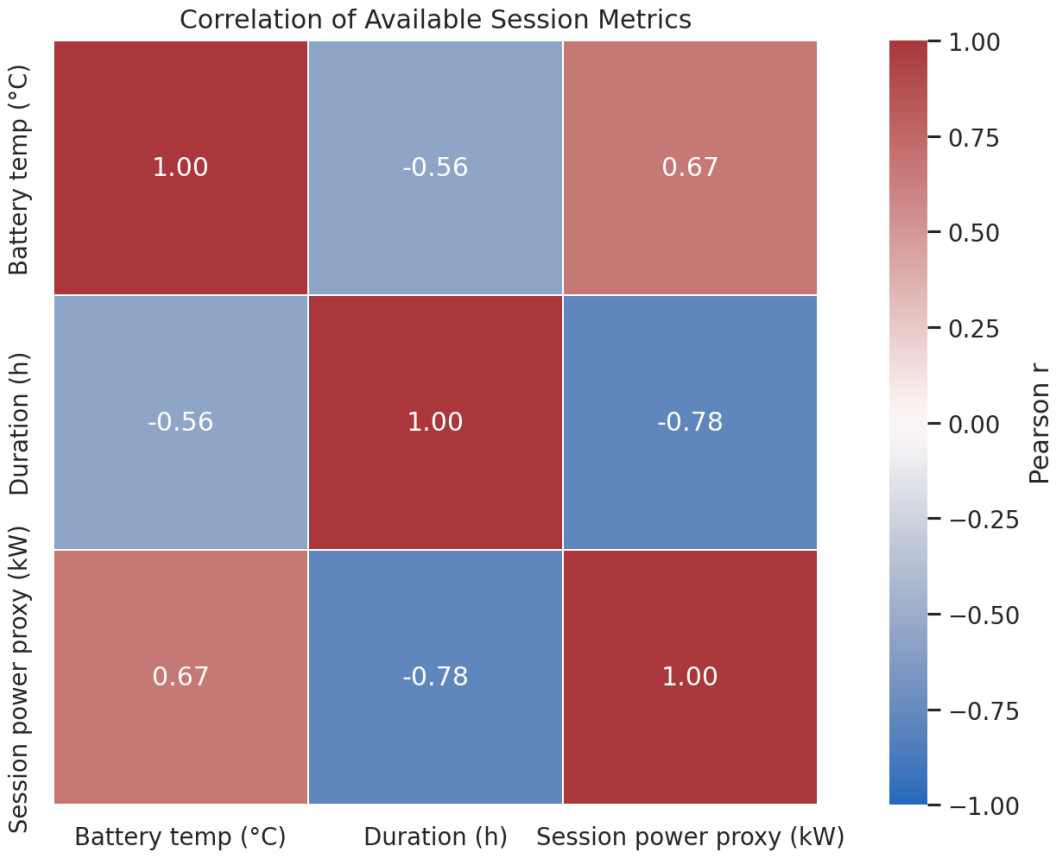


Figure 2. Correlation heatmap using fields available in the EV session dataset.

The revenue in the charts represents the modelled billing results declared in this report; the heatmap uses observable battery temperature, charging duration, and session power-rate proxy metrics. Because the input file lacks ambient temperature and grid-meter load fields, this report does not label these proxies as measured environmental data.

Section B Question 1

Production-grade data pipeline and engineered metrics

This project uses ev_campus_dataset.csv as the primary data for session-level energy analysis: among 1,200 records, Duration_Mins has 25 missing values and Energy_kWh has 35 missing values; the temperature column currently has no missing values. The 500 records in sales_data.csv are transaction-level sales data containing product, quantity, payment method, and date, suitable for revenue/product analysis, but they cannot be merged row-by-row with EV sessions to derive battery temperature or charging power. The provided cleaned_energy_sales_data.csv serves only as a reference for fields and existing modelled billing assumptions; the official output is re-cleaned from the raw EV CSV.

Cleaning / Feature	Rule
Energy_kWh	Impute with the group median by Charger_Type; fall back to the global median only if a group is entirely empty.
Duration_Mins	Impute with the column median; durations less than or equal to 0 are clipped to 1 minute.
Battery_Temp_C	Filter outliers below -20C or above 100C, and fill missing values with the column median.
Categorical fields	Validate AC/DC, Student/Staff/Visitor, Grid_Status_Label; illegal labels are kept as Unknown.
Energy and carbon	Power_Rate_kW = Energy_kWh / Duration_Hours; Carbon_Savings_kgCO2 = Energy_kWh x 0.475.
Modeled bill	Hourly rates AC=MYR 6, DC=MYR 15; 25% surcharge for 12:00-17:00; first 15 minutes of overstay are free, then MYR 0.50/minute.

The modelled billing parameters are derived from field patterns verifiable in the accompanying cleaned sample and do not represent real electricity tariffs approved by the university. Total bill = duration x corresponding hourly rate + peak surcharge + overstay fee. After cleaning, the 1,200 records have no remaining missing values; total energy delivered is 18,675.79 kWh, estimated carbon reduction is 8,871.00 kg CO2, and the total modelled bill is MYR 17,354.99. The carbon factor of 0.475 kg CO2/kWh is used according to the formula given in the assignment; the actual report requires the university to confirm the applicable grid factor.

Code Listing 3. Core feature engineering and median imputation

```
frame["Energy_kWh"] = frame["Energy_kWh"].fillna(
    frame.groupby("Charger_Type")["Energy_kWh"].transform("median")
)
frame["Duration_Hours"] = frame["Duration_Mins"] / 60.0
frame["Power_Rate_kW"] = frame["Energy_kWh"] / frame["Duration_Hours"]
frame["Carbon_Savings_kgCO2"] = frame["Energy_kWh"] * 0.475
frame["Base_Fee_MYR"] = frame["Duration_Hours"] * frame["Base_Rate_MYR"]
frame["Peak_Surge_MYR"] = np.where(
    frame["Start_Hour"].between(12, 17), frame["Base_Fee_MYR"] * 0.25, 0.0
)
frame["Overstay_Fee_MYR"] = (frame["Overstay_Mins"] - 15).clip(lower=0) * 0.50
frame["Net_Billable_MYR"] = (
```

```
    frame["Base_Fee_MYR"] + frame["Peak_Surge_MYR"] + frame["Overstay_Fee_MYR"]
)
```

Section B Question 2

Publication-quality visualization module

The Matplotlib and Seaborn module generates three main figures. The 'hourly grid demand and solar curve' required by the assignment lacks grid-meter load and solar-generation measurements in the source data, so the total session power rate per start hour is used as a load proxy, and a daylight bell curve explicitly labelled as synthetic is used to demonstrate the dual-line chart; this chart cannot be used for grid capacity decisions. The user-type box plot shows the modelled session cost distribution; the scatter plot shows the collected Battery_Temp_C and Energy_kWh with a superimposed linear trend.

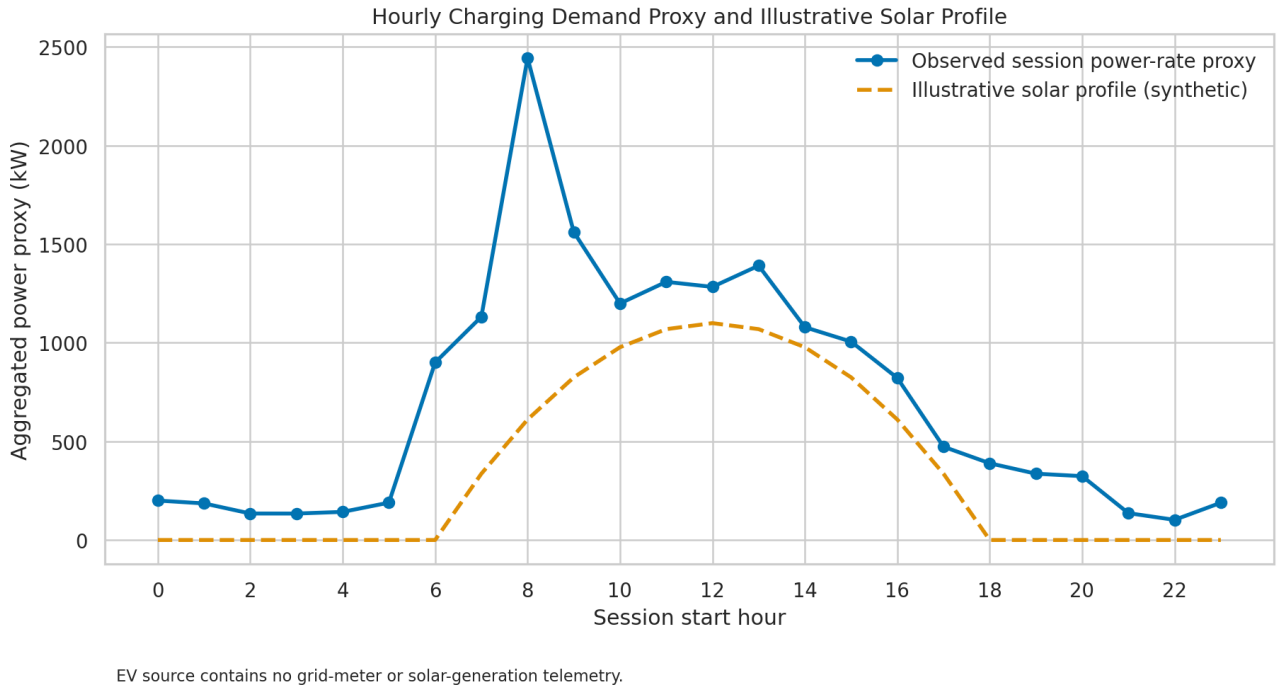


Figure 3. Requested demand/solar panel with measured-session proxy and synthetic solar profile.

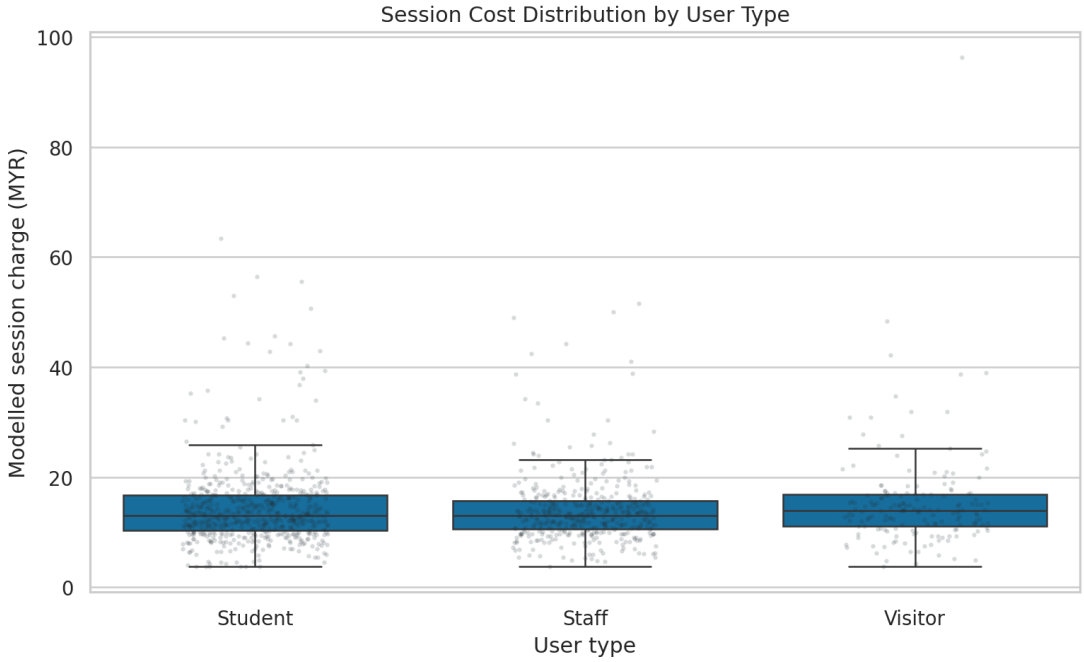


Figure 4. Modelled charging-session cost distributions by user type.

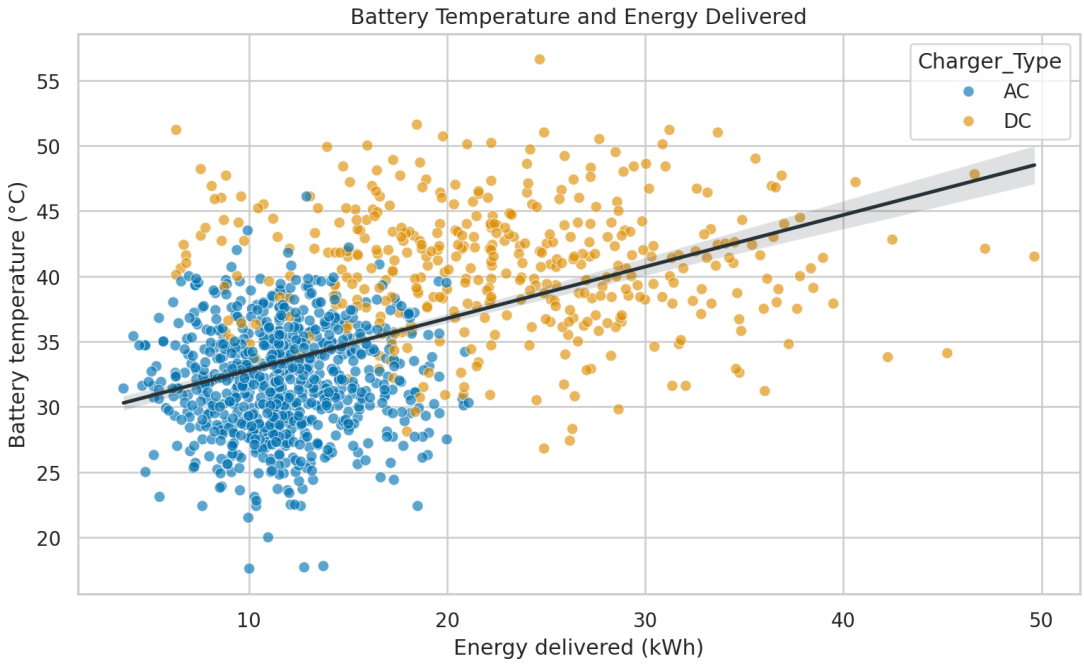


Figure 5. Battery temperature versus delivered energy with a fitted trend line.

In addition, Figure 1's hourly revenue-by-charger-type bar chart and Figure 2's available-metrics heatmap are generated, covering Section A's multi-bar chart and correlation heatmap requirements. Colors, labels, units, legends, and source limitations are directly annotated to avoid misreading modelled outputs as measured values.

Section B Question 3

Interactive Streamlit dashboard and operational controls

The Streamlit application provides four KPI cards, a demo date-range filter, a minimum-energy slider, a charger-type multiselect, a temperature-threshold slider, risk alerts, a hourly revenue chart, a power-rate proxy trend, a user-type box plot, a risk-record table, and filtered-result CSV download. File reading and cleaning functions all use st.cache_data to reduce repeated I/O on each widget interaction; data processing, feature engineering, and visualization are split into modules and include type hints, docstrings, logging, and exception handling (Streamlit, n.d.).

Code Listing 4. Cached loading, high-risk alert, and CSV export

```
@st.cache_data(show_spinner="Loading and cleaning EV session data...")
def load_and_clean_data(csv_path: str) -> pd.DataFrame:
    """Cache CSV loading and cleaning for responsive UI reruns."""
    return clean_sessions(read_sessions(csv_path))

high_risk = selected.loc[
    selected["Overstay_Mins"].gt(20)
    & selected["Battery_Temp_C"].gt(temp_threshold)
]
if len(high_risk) > 5:
    st.error("⚠️ Warning: large-scale grid overload risk detected; review DC charging power.")

csv_data = selected.to_csv(index=False).encode("utf-8")
st.download_button("Download filtered CSV", csv_data,
                   file_name="filtered_ev_sessions.csv", mime="text/csv")
```

The risk rule uses 'overstay > 20 minutes AND battery temperature > user-set threshold'; more than 5 records trigger an error banner, and 1-5 records display a warning. The management alert text specified in the assignment is kept in English in app.py. To satisfy the interaction requirement, the application's date filter maps records to demo dates from 2026-04-01 to 2026-04-20 in Session_ID order; the source file has no real dates, so the dashboard continuously reminds users that this is a demo dimension and should not be used for real time-series conclusions.



Figure 6. Captured console output from the data-cleaning, export, and chart-generation run.

How to run: install requirements.txt then execute streamlit run app.py. The execution evidence in the report is the actual data pipeline output; Streamlit was not pre-installed in this execution environment, so the interactive pages must be started after installing dependencies to view them.

Business Insights

At the current demo threshold, 24 records simultaneously satisfy overstay greater than 20 minutes and battery temperature greater than 40C; their average battery temperature is approximately 43.95C. The facilities team can first verify the equipment and alarm logs corresponding to these sessions, then assess whether the power ceiling for high-temperature DC equipment needs adjustment; the existing data lacks real-time grid meters and measured solar generation, so it is not appropriate to directly change campus-wide scheduling strategies based on it.

The session power-rate proxy peaks around 08:00, and hourly modelled revenue is also highest at 08:00. It is recommended to subsequently integrate hourly grid-meter load, solar inverter data, ambient temperature, and real start times to verify whether the morning peak creates grid pressure, and then design off-peak incentives or overstay management measures accordingly.

References

Matplotlib Development Team. (n.d.). Quick start guide. Matplotlib documentation. https://matplotlib.org/stable/users/explain/quick_start.html

NumPy Developers. (n.d.). Broadcasting. NumPy documentation. https://numpy.org/doc/stable/user/basics.broadcasting.html

pandas Development Team. (n.d.). Group by: split-apply-combine. pandas documentation. https://pandas.pydata.org/docs/user_guide/groupby.html

Python Software Foundation. (n.d.). functools — Higher-order functions and operations on callable objects. Python documentation. https://docs.python.org/3/library/functools.html

Seaborn Development Team. (n.d.). API reference. seaborn documentation. https://seaborn.pydata.org/api.html

Streamlit. (n.d.). Caching and data elements. Streamlit documentation. https://docs.streamlit.io/develop/concepts/architecture/caching and https://docs.streamlit.io/develop/api-reference/data

Taylor's University. (2026). ITS72604 final examination: September 2026 semester [Examination paper].

Implementation Files

The editable implementation is provided alongside this report. The project bundle includes the raw EV session CSV, generated cleaned CSV, application, cleaning pipeline, visualization module, dependency list, chart images, and a run log. The separate sales_data.csv is not merged into the EV session table because it represents a different transaction grain and subject area.

File	Purpose
app.py	Interactive Streamlit dashboard.
energy_pipeline.py	CSV validation, cleaning, imputation, feature engineering, and export.
visualizations.py	Five labelled chart outputs.
requirements.txt	Python dependencies for local execution.
data/ev_campus_dataset.csv	Raw source dataset used by this report.
data/cleaned_energy_sales_data.csv	Cleaned output and derived session metrics.

Submission Links

GitHub URL: https://github.com/sadboy66684/final-assessment-liyuzhou

Video Demonstration URL: [To be provided by the candidate]

The GitHub repository contains the full source code, data files, generated figures, and this report in both DOCX and PDF formats.